

Module 7 – Code Quality and SonarQube

1. Introduction to SonarQube

What is SonarQube?

SonarQube is an open-source static code analysis tool used to inspect source code for bugs, vulnerabilities, code smells, duplicate code, and maintainability issues. It supports multiple programming languages such as Java, Python, JavaScript, C#, C++, PHP, Kotlin, and Go.

Purpose of SonarQube

- Detect bugs before deployment.
- Identify security vulnerabilities.
- Improve code quality and maintainability.
- Enforce coding standards.
- Reduce technical debt.

Benefits of SonarQube

- Detects bugs early.
- Improves software quality.
- Reduces maintenance cost.
- Finds duplicate code.
- Measures code complexity.
- Generates detailed reports.
- Integrates with Maven, Gradle, Jenkins, and CI/CD tools.

Clean as You Code Principle

The "Clean as You Code" principle focuses on maintaining the quality of newly written or modified code instead of fixing all existing issues. Every new piece of code should be clean, secure, and maintainable.

2. Static Code Analysis vs Runtime Testing

Static Code Analysis

Static code analysis examines source code without executing the program. It identifies coding issues, security vulnerabilities, duplicate code, and coding standard violations.

Examples: SonarQube, PMD, Checkstyle, SpotBugs.

Runtime Testing

Runtime testing evaluates the application while it is executing to detect logical errors, runtime exceptions, and functional issues.

Examples: JUnit, TestNG, Selenium.

Static Code Analysis	Runtime Testing
No program execution	Program execution required
Finds code quality issues	Finds runtime errors
Faster	Slower
Example: SonarQube	Example: JUnit

3. SonarQube Architecture

SonarQube Server

The central application that receives analysis reports, manages users, and displays dashboards.

Compute Engine

Processes analysis reports, calculates metrics, evaluates quality gates, and updates the database.

Search Server (Elasticsearch)

Provides fast searching, indexing, and filtering of projects and issues.

SonarQube Client

Sends project analysis results to the SonarQube Server using tools like Maven, Gradle, or Sonar Scanner.

Sonar Database

Stores projects, issues, metrics, users, quality profiles, quality gates, and analysis history.

4. Quality Profile

A Quality Profile is a collection of coding rules applied during analysis.

Examples:

- Avoid unused variables.
- Avoid duplicate code.
- Do not leave empty catch blocks.
- Follow Java naming conventions.

5. Quality Gate

A Quality Gate is a set of conditions used to determine whether a project satisfies the required quality standards.

Typical conditions:

- Code Coverage > 80%
- Critical Bugs = 0
- Vulnerabilities = 0
- Duplicate Code < 3%

If all conditions are satisfied, the project passes the Quality Gate.

6. Integrating SonarQube with Maven

Add SonarQube Maven Plugin

```
<build>
  <plugins>
    <plugin>
      <groupId>org.sonarsource.scanner.maven</groupId>
      <artifactId>sonar-maven-plugin</artifactId>
      <version>3.11.0.3922</version>
    </plugin>
  </plugins>
</build>
```

Configure SonarQube Server URL

```
sonar.host.url=http://localhost:9000
```

Generate Project Token

1. Login to SonarQube.
2. Go to **Administration** → **Security**.
3. Generate a project token.
4. Copy the generated token.

Run Sonar Analysis

```
mvn clean verify sonar:sonar
```

This command cleans the project, builds it, runs tests, and uploads the analysis report to SonarQube.

7. Viewing SonarQube Dashboard

The dashboard displays:

- Bugs
- Vulnerabilities
- Code Smells
- Code Coverage
- Duplicate Code
- Reliability Rating
- Security Rating
- Technical Debt

8. SonarQube Code Analysis

Duplicate Code

Duplicate code refers to repeated code blocks within a project. It increases maintenance effort and the possibility of defects.

Cyclomatic Complexity

Cyclomatic Complexity measures the number of independent execution paths in a program.

- 1–10 : Low Complexity
- 11–20 : Moderate Complexity
- Above 20 : High Complexity

Higher complexity makes code harder to test and maintain.

Code Smells

Code Smells are poor programming practices that reduce code readability and maintainability.

Examples:

- Long methods
- Large classes
- Duplicate code
- Unused variables

- Poor naming conventions

Unit Test Coverage

Code Coverage measures how much of the source code is tested by unit tests.

Higher coverage indicates better software reliability.

Coding Standards

SonarQube checks whether the project follows standard coding practices such as:

- Proper variable naming
- Consistent indentation
- Meaningful comments
- Proper exception handling

Potential Bugs

Examples:

- Null Pointer Exception
- Division by zero
- Resource leaks
- Infinite loops
- Unreachable code

Security Vulnerabilities

SonarQube detects:

- SQL Injection
- Cross-Site Scripting (XSS)
- Hardcoded passwords
- Weak encryption
- Unsafe file handling

Comments

Comments should explain **why** the code is written rather than simply describing what the code does.

9. Interpreting the Quality Gate

Passed

- No critical bugs
- No security vulnerabilities
- Good test coverage
- Acceptable duplicate code

Failed

- Critical bugs detected
- Low code coverage
- Excessive duplicate code
- Security vulnerabilities found

10. Reading Sonar Logs

Run the following command:

```
mvn clean verify sonar:sonar
```

Successful execution displays:

```
BUILD SUCCESS
```

The logs contain information about:

- Project analysis
- Files analyzed
- Issues detected
- Analysis upload status
- Quality Gate result